Contents

Volume 06 — NLP, Transformers, LLMs (L54–L63) **1**
L54: Foundations of natural language processing 1
L55: Sequential modeling with RNNs and LSTM 2
L56: Evolution from LSTMs to transformers 3
L57: Transformer architecture — encoder 4
L58: Practical exercise 1 — RNN vs LSTM sentiment classification 5
L59: Transformer architecture — decoder 6
L60: BERT 7
L61: GPT — decoder-only large language model 8
L62: Prompt engineering basics 9
L63: Hands-on on LLM 10

Volume 06 — NLP, Transformers, LLMs (L54–L63)

Generative AI & LLMs · NPTEL · Generative AI / LLMs **notes-3** — concept exam notes (definitions & what things do). Breadth over lab depth.

L54: Foundations of natural language processing

Learning focus

- Define NLP as mapping human language to something a model can compute, then to an action.
- Walk the pipeline: **raw text → preprocessing → numerical representation → model**.
- Tokenize at word / sentence / **subword** / character granularity; apply stop-word removal and **lemmatization**.
- Contrast **Word2Vec** (local windows, king–queen arithmetic) with **GloVe** (co-occurrence matrix).
- List NLP challenges that motivate **sequential** models in Lec 55 (especially **long-range dependency**).

Exam lines

- NLP turns language into **clean tokens**, then **vectors**, then a task output (spam, sentiment, translation, QA, …).
- Preprocessing: lowercase, strip punct, **tokenize**, drop stop words, **lemmatize**.
- Word2Vec: local windows + vector arithmetic (king − man + woman ≈ queen).
- GloVe: **co-occurrence** counts → dense embeddings.
- Discrete, drifting, ambiguous language plus **long-range** binding still break bag-of-features thinking — hence sequence models next. Contextual BERT/GPT embeddings wait for the transformer week.

Concept tables

Input	Output	Action / task
“Is this email spam?”	Yes	**Spam detection**
“The movie was amazing”	Positive	**Sentiment analysis**
“Hello”	“Hola”	**Machine translation**
“What is generative AI?”	A definition paragraph	**Question answering**
Stage	What happens	Example
——-	——————-	———
Raw text	User text, or speech converted to text	Full sentence + punctuation
Text preprocessing	Clean, normalize, drop extras	Content words such as **absolutely / love / movie**
Text representation	Words → **numerical vectors** (models do not read strings)	Embedding per token
Model	Learns patterns from those vectors	Sentiment model
Output	Task prediction	**Positive** sentiment

L55: Sequential modeling with RNNs and LSTM

Learning focus

- Motivate sequence models from **word order** (“dog bites man” vs “man bites dog”).
- Distinguish **short-term** vs **long-term** temporal dependence.
- Write the RNN **hidden-state** and **output** updates; map **one-to-one / one-to-many / many-to-one / many-to-many**.
- Explain **vanishing / exploding** gradients and **BPTT** as why vanilla RNNs fail on long gaps.
- Walk LSTM **forget, input, output** gates and the **cell state** highway.

Exam lines

- Order is meaning: same bag of words, different sequence, different sentence.
- Feedforward nets have **no** sequential memory; RNNs store it in $h_t = \tanh(Vh_{t-1} + Ux_t + b_h)$.
- RNN patterns match captioning, sentiment, translation / tagging.
- Long gaps + BPTT → **vanishing / exploding** gradients.
- LSTM adds **cell state** c_t and three gates so memory can survive many steps.
- **Still not enough** when dependencies are very long → **transformers / attention** in Lec 56.

Concept tables

Sentence	Meaning
The dog bites man	Dog is the biter
Man bites dog	Same words, **different** event
Matrix	Connects
——–	———-
U	Input x_t → hidden h_t
V	**Recurrent:** previous hidden h_{t-1} → current h_t
W	Hidden h_t → output $\hat{y}_t$

L56: Evolution from LSTMs to transformers

Learning focus

- Place RNN → LSTM → **attention (2014)** → **transformers (2017)** on the lecture's timeline.
- Explain why recurrence is expensive for long documents ($O(n)$ sequential steps, memory of every h_t).
- Define **encoder–decoder attention** (context $c_t = \sum_j \alpha_{tj} h_j$) and **self-attention** (Q, K, V).
- Motivate **multi-head** attention and **causal masks** ($-\infty$ → softmax 0).
- Contrast RNN/LSTM vs transformers: sequential vs **parallel**, hidden state vs **direct** token links.

Exam lines

- Recurrence forces $O(n)$ hops and a stack of hidden states; long documents (paragraph-scale) become expensive and forgetful.
- Attention **reweights** encoder states (or tokens) so it can lock onto animal, not the whole sentence equally.
- Self-attention: Q, K, V from the **same** sequence; multi-head = several relations at once; **masks** implement causal decoding.
- Transformers (2017) combine attention with **parallel** reads of the whole sequence — the reason they replace LSTMs as the default sequence backbone. Next lecture: **transformer architecture** in detail.

Concept tables

When	What
~2013	**RNNs** in wide NLP use
1997 / adopted ~2014	**LSTMs** (Hochreiter & Schmidhuber 1997; popular after 2014)
2014	**Attention** (the ingredient transformers will rest on)
2017	**Transformers** — NLP landscape changes; leads to **BERT, GPT, LLaMA**, ...

When	What
Vector	Informal job (lecture)
——–	————————-
Query	What am I looking for?
Key	What information do I contain?
Value	What should I pass forward?

L57: Transformer architecture — encoder

Learning focus

- State why RNNs, LSTMs, and GRUs motivated the 2017 transformer: sequential compute, hard parallelization, and remaining recurrence after attention.
- Describe the encoder as **six identical stacked layers**, each with **multi-head self-attention** and a **position-wise feed-forward network**.
- Explain why **sinusoidal positional encodings** are added to token embeddings when every token is processed at once.
- Compute a $d_{\text{model}} = 4$ positional encoding for the token sat in "the cat sat on the mat."
- Write residual + layer-norm and the two-linear ReLU FFN used in the original paper.

Exam lines

- Seq2seq recurrence made training slow, blocked parallelization, and still struggled with **very** long range; Attention Is All You Need (2017) dropped recurrence and convolutions.
- The encoder is **six identical layers** (different weights): multi-head self-attention + position-wise FFN, each wrapped in residual Add & Norm.
- Encoder **understands** all input tokens at once; decoder **generates** one token at a time from past outputs **and** encoder context (English→German).
- Simultaneous encoding loses order, so **sinusoidal PE** is added to embeddings; even dims sine, odd dims cosine.
- Residual $x+\text{Sublayer}(x)$ keeps information from being wiped; layer-norm stabilizes training; FFN is $512 \to 2048 \to \text{ReLU} \to 512$.

Concept tables

Limitation	What happens
Sequential computation	Word 1, then 2, then 3, … Training is slow.
Hard to parallelize	Later tokens wait on earlier hidden states; many operations cannot run together.
Very long-range dependency	LSTMs help, but words many sentences apart are still a problem.
Block	Job

Limitation	What happens
——-	—–
Encoder	**Understand** the English input. Process **all input tokens simultaneously**. Emit a **contextual representation** of every token.
Decoder	**Generate** the German translation. At **inference**, it does **not** emit all words at once: **one word at a time**.

L58: Practical exercise 1 — RNN vs LSTM sentiment classification

Learning focus

- Implement **binary sentiment classification** on **IMDb** with a **SimpleRNN** and an **LSTM** in TensorFlow / Keras.
- Pad / truncate reviews to a fixed length with **post-padding** and **post-truncation**.
- Compare validation accuracy/loss, test accuracy, and confusion matrices.
- Inspect **LSTM hidden-state** heatmaps (tanh activations) and decode two test reviews.

Exam lines

- IMDb 25k/25k, vocab **10k**, length **200**, **post-pad**; Embedding **128**, recurrent **64**, sigmoid + **Adam** + **BCE**, 10 epochs, batch 64, 20% validation.
- SimpleRNN stuck near **50%**; LSTM near **85% val / 82% test** because of **forget / input / output** gates and a **cell state**.
- Hidden-state heatmaps show tanh activity on real tokens, then **constant** rows on **padding**.
- This exercise is **sentiment RNNs**, not a transformer; transformer training/inference is Lec 63.

Concept tables

Split	Reviews
Train	25,000
Test	25,000
Total	50,000
Parameter	Value
———	——:
`vocab_size` / `num_words`	**10,000**
`maxlen`	**200**

L59: Transformer architecture — decoder

Learning focus

- List the decoder's **three** sub-layers and why the encoder only needed two.
- Explain **masked** multi-head attention vs **encoder-decoder** attention (Q from decoder, K, V from encoder).
- Reconstruct **BPE** (frequency) vs **WordPiece** (likelihood) as the pre-processing step before embeddings.
- Recall the **WMT 2014** En–De / En–Fr training setup and transformer **advantages vs quadratic cost**.

Exam lines

- Decoder layer = **masked self-attention** + **encoder-decoder attention** (Q decoder, K, V encoder) + **FFN**, each with residual Add & Norm; **six** layers in the paper.
- Masking stops future-target leakage; cross-attention injects source context; softmax over the vocab emits the next word.
- Extra sub-layer exists because generation has **two** look-ups (past outputs **and** encoder).
- BPE (frequency) and WordPiece (likelihood) shrink vocab and handle unknowns (cyber+security).
- Trained on WMT 2014 En–De (~4.5M, BPE) and En–Fr (~36M, WordPiece); pay $O(n^2)$ attention cost.

Concept tables

Side	Job	Attention it needs
Encoder	Only **understand** the input	Self-attention over the source + FFN (**two** sub-layers)
Decoder	**Two** jobs	(1) Look at **already generated** words → **masked** self-attention. (2) Attend to **encoder** context → **cross-attention**. Then FFN.

Model (as stated)	Heads
———————-	——:
Original transformer	**8**
Transformer **big**	**16**

L60: BERT

Learning focus

- Contrast BERT with **static** embeddings (Word2Vec / GloVe) and **left-to-right** LMs using the lecture's bank and movie was — examples.
- Explain why BERT is **encoder-only**: **understand** language, not generate long text.
- Describe the two pre-training tasks **MLM** and **NSP**, including the **15% / 80–10–10** masking recipe.
- Trace input construction: **token + segment + position** embeddings, `[CLS]`, `[SEP]`.
- Distinguish **self-supervised pre-training** from **supervised fine-tuning**, and name the BERT **variants** listed in class.

Exam lines

- Static bank vectors and left-to-right LMs miss context; BERT is a **bidirectional encoder** LM (Google / Devlin et al.).
- Encoder-only because target tasks need **labels and short answers**, not free generation.
- Pre-train **MLM** (15% selected; 80% `[MASK]`, 10% random, 10% unchanged) and **NSP** (50/50 IsNext) on BooksCorpus + Wikipedia with WordPiece.
- Input = token + position + segment; `[CLS]` for classification; fine-tune with a small head on labeled data.
- Variants: RoBERTa, ALBERT, DistilBERT, ELECTRA — still BERT-family encoders, not GPT.

Concept tables

Task	Input (lecture)	Output
Sentiment	I absolutely love this movie.	**positive** (one label, not a sentence)
NER	Dr. Joan visited Ames Hospital on August 1st, 2025.	person / organization / date
QA	Paragraph: Paris is the capital of France. Q: What is the capital of France?	**Paris**
Text classification	(same idea)	class label

Stage	School analogy	Data
Pre-training	KG through class 12: many subjects	**Massive unlabeled** corpus
Fine-tuning	Medical school: specialize	**Smaller labeled** set

L61: GPT — decoder-only large language model

Learning focus

- Contrast **BERT (encoder, understanding, MLM)** with **GPT (decoder, generation, next-token)** as taught in this pair of lectures.
- Explain **causal / masked self-attention** and why a **bidirectional encoder cannot** generate without **information leakage**.
- Walk the **autoregressive** loop on The capital of France is … Eiffel Tower.
- Describe GPT's **pre-train (unlabeled next-word) → supervised fine-tune** framework and the **decoder-only** block (no encoder–decoder attention).
- List GPT-1 vs GPT-2 scale numbers and the **limitations** named here (including **hallucination**). Prompt recipes are Lec 62 — not this video.

Exam lines

- GPT-1 (OpenAI / Radford, 2018) is **decoder-only**; BERT is **encoder-only**. GPT predicts the **next** token; BERT predicted **masks**.
- Causal masking enables generation; bidirectional encoder attention would **leak** future tokens.
- Generation is **autoregressive**: append Paris, then `.`, then It is known for the Eiffel Tower.
- Pre-train unlabeled next-word (12 decoder blocks, no cross-attention) → fine-tune with small linear heads (classification, entailment, similarity, MCQ).
- GPT-1 → GPT-2: 117M→1.5B parameters, 5→40 GB, 512→1024 context, 40k→50k BPE. Failures named here: hallucination, bias, cost, context, prompts, cutoff, safety — not RAG/ethics weeks.

Concept tables

Year (lecture)	Model
2018	**GPT-1** (same year as BERT)
2019	**GPT-2**
2020	**GPT-3**
2022	**GPT-3.5**
2023	**GPT-4**
2024	**GPT-4o**
2025	**GPT-5**, and the lecture's latest mention **GPT-5.6**
	BERT
–	———-
Aim	Language **understanding**
Stack	**Encoder-only** transformer
Pre-train prediction	**Masked** token (both sides): the boy is **[MASK]** football → playing

L62: Prompt engineering basics

Learning focus

- Define a **prompt** as the input that **guides** an LLM, and **prompt engineering** as designing that input so the answer matches what you asked for.
- List what a prompt can contain: **instructions, questions, context, constraints, examples**.
- Name the six techniques taught: **zero-shot, one-shot, few-shot, chain-of-thought, role, structured-output**.
- Contrast a **vague** vs **specific** prompt using the explain machine learning pair.

Exam lines

- A prompt is the **guiding input**; engineering it means instructions + context + constraints + examples so the LLM matches intent.
- Good prompts are clear, contextual, specific, audience- and format-aware; avoid vagueness, missing context (hallucinated background), and unrelated question piles.
- Six named techniques: **zero-shot** (no demos), **one-shot**, **few-shot** (2–5), **chain-of-thought** (step-by-step $5/$2 shop), **role** (physics professor / $F = ma$ cart), **structured output** (JSON / bullets / code).
- Explain machine learning vs 150 words, first-years, one example is the running quality contrast.
- RAG, ethics, and LoRA are **not** in this transcript.

Concept tables

Ingredient	Lecture example
Instructions	Summarize this article in **five bullet points**
Context	The article itself, pasted in
Questions	Follow-ups (e.g. weather in Bangalore over five months **from that summary**)
Constraints	Avoid rainy season; only summer/winter/…
Examples	Demonstrations of the desired pattern
Role of the prompt	What it does
———————–	————–
Define the task	Summarization, translation, explanation, classification, QA, **code generation**, …
Provide context	Background so the situation is understood
Specify the audience	Explain AI to a **layperson** vs **PhD / research** scholars → **depth** changes
Control format	Bullets, **table**, code, essay; “three bullet points” vs a long paragraph
Reduce ambiguity	Task + context + audience + format + extra specifics → fewer misreads

L63: Hands-on on LLM

Learning focus

- Implement **scaled dot-product attention** with a **causal (triangular) mask** and **next-token** training (input shifted by one).
- Train a **tiny decoder-only** transformer from scratch on a **character** corpus (paragraph × 80).
- Decode with **greedy**, **top-k**, and **nucleus (top-p)** sampling; see **temperature** and **seed**.
- Compare **character** vs **BPE / WordPiece** on Hugging Face tokenizers; run **DistilGPT-2** as a pretrained **completion** model (not instruction-tuned chat).
- Treat **prompts** as incomplete context. This notebook does **not** implement RAG.

Exam lines

- Causal LMs learn **next-token** prediction; **causal masks** block future tokens in train and decode.
- Tokenization is **model-specific**: use the **same** tokenizer at train and inference.
- **Greedy** is deterministic (seed-invariant); **top-k** and **nucleus** add **controlled** diversity.
- Prompt wording **shifts** the continuation distribution; a prompt is **unfinished context**, not a separate RAG index.
- Tiny char-LM (block 64, Adam 3×10^{-4}, loss ~3.6→0.16) plus DistilGPT-2 decoding is the whole lab — **no RAG, no ethics module**.

Concept tables

Hyperparameter	Value	Lecture name
Block size	**64**	context window (64 **characters** per sample)
Batch size	32 (from printed shapes)	independent sequences per step
Split	**90%** train / **10%** val	prefix of the token stream vs rest

Mode	Rule
——	——
Greedy	Always take the **max** probability token. **No stochasticity** — same string every time (seed / temperature do not change it).

Hyperparameter	Value	Lecture name
Top-k	Keep the k **largest** logits; sample from that set (with temperature, e.g. 1).	
Nucleus / top-p	Smallest prefix of the ranked vocab whose probabilities **sum to ≥** p (e.g. **0.9**); sample from that set.	